

Real-Time Operating System for Multitasking Control in the Robotics and Automation Industry

Javed Sayyad*

Robotics & Automation Department,
Symbiosis Institute of Technology(SIT),
Symbiosis International (Deemed
University)(SIU),
Lavale, Pune, Maharashtra, India.
jksayyad23@gmail.com

Ashwini Jatti

Computer Science and Engineering,
Symbiosis Institute of Technology(SIT),
Symbiosis International (Deemed
University)(SIU),
Lavale, Pune, Maharashtra, India.
ashwini.jatti@sitpune.edu.in

Khush Attarde

Robotics & Automation Department,
Symbiosis Institute of Technology(SIT),
Symbiosis International (Deemed
University)(SIU),
Lavale, Pune, Maharashtra, India.
khush.attarde.btech2021@sitpune.edu.in

Ramesh B.T

Robotics & Automation Department,
Symbiosis Institute of Technology(SIT),
Symbiosis International (Deemed
University)(SIU),
Lavale, Pune, Maharashtra, India.
ramesh.bt@sitpune.edu.in

Shubhangi Deokar

Computer Science and Engineering,
Symbiosis Institute of Technology(SIT),
Symbiosis International (Deemed
University)(SIU),
Lavale, Pune, Maharashtra, India.
shubhangi.deokar@sitpune.edu.in

Abstract— Real-Time Operating System (RTOS) is an Operating System (OS) that does a task in a constrained time. The RTOS is integrated with the microprocessor or microcontroller for specialized applications where the time-bound completion of the task is very much substantial. ESP32 is a microcontroller that comes with a package of advantageous and suitable features for RTOS. The ability of equipment like motors, sensors, and actuators to multitask is critical in the Robotics and Automation (R&A) industry. In industries, a dedicated/centralized system allocated for a particular activity is not economically feasible; yet, when performing several tasks, some delay or lag is introduced in the task. Consequently, the proposed solution for industrial multitasking is cost-effective and efficient. The RTOS is suitable for machines constrained to do painstaking work, like a robotic arm or CNC drilling. In the R&A industry, a hardware interruption is often critical for an emergency, which may also be accomplished using RTOS. In such applications, implementing a software timer is also essential to delay a task for a period of time. Stepper and servo motors are widely employed in the R&A industries for various applications. This article presents the multitasking of four stepper motors and the introduction of a software timer, which will be developed when the stepper motor multitasking is implemented. For multitasking, the FreeRTOS library will be utilized.

Keywords— Real-Time Operating System, FreeRTOS, Espressif32 (ESP32), Software timers, Stepper motor, multitasking

I. INTRODUCTION

Multitasking of devices such as various motors (actuators) and sensors is critical in Robotics and Automation (R&A) industry. A high-performance CPU and Operating System (OS) are necessary for complex computational multitasking without delay in the industry. A Real-Time Operating System (RTOS) is an OS designed to quickly complete tasks with minimal delay, enhancing the industry's efficiency. In the R&A industry, one robot does specific work multiple times, giving an example of a CNC machine used for milling or drilling so that the drilling hand will move. So, such a task can be quickly done using RTOS. The ESP32 is an excellent microcontroller for RTOS since it has a powerful CPU and many other capabilities. Stepper motors are considered since they are commonly used in the R&A industry to multitask. The ESP32 and the FreeRTOS API library [1] are utilized to control and multitask the four stepper motors.

The OS is the backbone of any computer, micro-computer, and micro-controller and is essential for handling computer hardware and software integration and task synchronization. It also provides an interface between the user and the computer hardware by providing Graphical User Interface (GUI). The fundamental function of OS is the execution of the program, memory management, input and output management of the devices and handling the interrupts. Most of the OS differs in the program scheduling process. Some commonly used examples of OS are Windows OS, Linux OS, and Mac OS. There are different types of OS, such as batch OS, time-sharing OS, networking OS, distributed OS, and RTOS. In the OS, the essential part is program scheduling and interrupt management.

Process scheduling is a process manager activity that removes a running process from the CPU and selects other processes based on a specific strategy. Therefore, scheduling is assigning the resources in the required order to perform the tasks. So, such OS allows multiple processes to be loaded into executable memory simultaneously, and the loaded processes share the CPU using time-division multiplexing. So, there are different types of program schedules based on function, like priority-based scheduling, pre-emptive scheduling, non-pre-emptive scheduling, and entirely fair scheduling. The RTOS is much more focused on priority-based scheduling and pre-emptive scheduling. RTOS is different from other standard OS in terms of many functions like time deterministic, strict deadlines, interrupt handling etc.

RTOS supports slow and fast context switching, demonstrating its ability to switch tasks when time is limited [2]. It improves the performance of all devices, hence increasing the output of all available resources. RTOS effectively manages memory allocation, but the structuring of working algorithms is quite complex. Responding to interrupt signals requires specific drivers so the system can reply rapidly. RTOS may be used for various applications that take advantage of the aforementioned core features, such as robotics applications and embedded real-time systems. Examples of software include FreeRTOS, VxWorks, and LynxOS, among others.

FreeRTOS is part of the RTOS, designed to be so small to run on a microcontroller, though the applications are not just limited to microcontrollers. The microcontroller is a resource-constrained processor with a single chip, processor, Read

Only Memory (ROM), and Random Accessory Memory (RAM). It also includes various input-output pins. The program we need to work on is stored in the ROM and executed from ROM only. RAM, we used when it is required by the programs while executing. FreeRTOS provides real-time scheduling and communication between tasks. More features can include timing and synchronization primitives; we can also say it is a Real-time kernel and a real-time executive. It can also have features like a command console interface or network stacks which can be integrated with add-on components. Microcontrollers are used in an embedded system where the job is specified. The ESP32 is a low-cost system [3] and a suitable microcontroller board that can perform the RTOS using the FreeRTOS library. A comparison between ESP32 and STM32 can be seen in Table 1.

Section 2 discusses the proposed system, which includes the RTOS architecture and hardware architecture. Section 2 presents the proposed system architecture followed by the proposed system programming and methodology. Section 3 includes the results and discussion of our findings from this proposed methodology. Section 4 is the conclusion obtained from this paper, and then references are included in Section 5.

II. PROPOSED SYSTEM

The proposed system block diagram can be seen in Fig. 1. Here, the ESP32 microcontroller is loaded with the RTOS. The stepper motor driver is connected to the ESP32, and a stepper motor is then operated by attaching it to the driver.

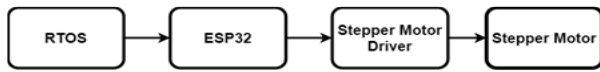


Fig. 1. Proposed system block diagram

A. RTOS Architecture

The main parts of RTOS are memory management, task scheduling, synchronization and communication, interrupt management, event handling, and timer management [4]. The RTOS architecture can be seen in Fig. 2.

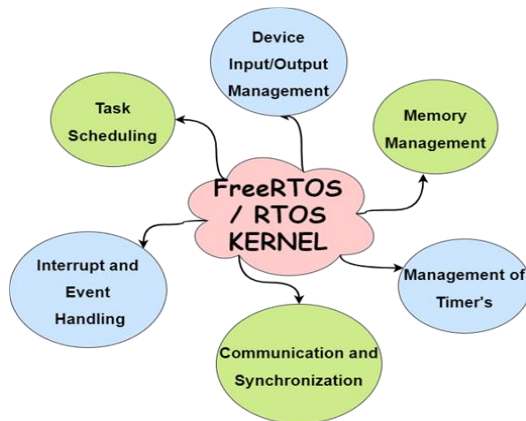


Fig. 2. RTOS kernel architecture

a) Task Scheduling

A task is a sequential line of code. The Task Scheduling in the RTOS is Priority and Pre-emptive based. Pre-emptive scheduling means tasks or processes are given within a specific time. The processes can be completed in this time, but allotted time matters here. After that task will be removed from the execution. In Pre-emptive Scheduling, if CPU burst time is left, the task is put back in the ready queue. The task

will be going to be there in the ready queue until the next opportunity for execution of it. Also, the priority of processes matters, meaning a task or process with a lower priority will be pre-empted by a higher priority. In RTOS, the hardware interrupt is given the highest priority. It is unpredictable when a task will be going to be suspended or when it is going to be resumed.

Suppose a task is interrupted before executing an instruction. In that case, the other tasks will execute, and it will change the processor register values, so when an interrupted task is resumed, it is unaware that the processor register values are changed and hence after performing the instructions, it results in incorrect final values. So, to avoid such errors, it should have the same context after resuming and before the task interruption. So, the OS kernel is responsible for ensuring the saving of the context before the suspension, and whenever the task resumes, it restores the saved context. This process of saving the context of a suspended task and restoring the context of the resumed task is called context switching. In a single core system, the task must be divided into time slices to run the task simultaneously. The scheduler in the OS should decide which tasks should be run in each time slot.

TABLE 1. COMPARISON TABLE OF ESP32 AND STM32

Features	ESP32	STM32
Processor	Tensilica Xtensa LX6 32-bit	32-bit ARM cortex M3
Core of processor	Dual-core & Single-core	Single-core
Flash memory space	4 MB	128 Kb
RAM	512 Kb	4 to 96 Kb
On-chip wireless connectivity	Supports	Not support
RTC memory	16 Kb	No

The time slice in a single-core system is known as ticks in the RTOS. The default value of the time slice in an RTOS is 1 ms. A hardware timer is used to make interrupts every 1 ms, and ISR for the hardware timer runs the scheduler to choose the following task. Tasks with the highest priority are given respect to every tick interrupt, and if the highest priority is the same for two tasks or processes, then a round-robin algorithm manner is applied. In round-robin scheduling, each task is allocated with an equal share of CPU time so that a task is in a circular queue, and when an allotted CPU time expires, the task is put at the end of the queue, and another task is taken from the queue [5].

This terminology of Round-robin scheduling is complicated in RTOS, but as we said earlier, when the priorities of tasks are identical, we can utilize such fashion in an RTOS. It is important to note that if the highest priority task becomes ready, it needs to be executed without waiting for the next tick. The same priority tasks can be implemented or executed simultaneously using the multicore system, which means a dual-core system in ESP32. In RTOS, the Idle tasks are created automatically when the RTOS scheduler starts, ensuring that at least one task is always running. When all tasks are blocked, there should be at least one task in the running state. An idle task is created with the lowest possible priority to ensure that the CPU time is not consumed if the higher priority task is in the ready state [6] [7]. The idle task is only essential for freeing the allocated memory by the RTOS.

b) Interrupt and Event Handling in RTOS

The hardware interrupt has the highest priority in most microcontrollers, even in the ESP-32, until and unless the user disables it. For the R&A industry, hardware interrupts are essential. These interruptions or events occur synchronously, not part of the running program. These types of interrupts are significant for stopping the CPU from doing its function and tend to perform other functions, and it is only known as ISR. Such types of interrupts can be done by using the push button, key pressing or changing the voltage in the input pins, hardware timer expiration, communication buffers, etc. Interrupt handling is also known as Interrupt Service Routine (ISR).

It interrupts the execution of the code. ISRs can handle hardware timer overflows, pin state changes, new communication on the bus, etc. ISRs are part of the program code that can control the interrupt request; here, the interrupt can be either hardware or software. So, an Interrupt Service Routine (ISR) is a software routine called or invoked by the hardware in response to the interrupt. ISR examines, handles and processes the interrupts and returns the logical interrupt value, and after completion of the processing of interrupts, the ISR notifies the OS kernel about the return value. The event handling is there to handle the software interrupts, whereas the interrupt handles are there for hardware events.

c) Memory Management in RTOS

In low-level languages like C and C++, it tracks the memory usage in a system in both Random Accessory Memory (RAM), which means volatile memory and non-volatile memory like a flash, and in RTOS, this task increases the difficulty as well as complexity. Volatile memory, like RAM, usually divides into static, stack, and heap. Static memory stores the global variables. The stack is organized in the Last-in-First-Out (LIFO) system so that the function variable can be pushed onto the stack whenever a function is called or returns to the first function. Then that function variable can be hidden and move forward to the execution of the function where it left off. Here heap memory is allocated at a run time, and the user needs to assign it. That's why heap memory allocations are known as dynamic memory allocations. A task or object is stored in the heap memory, whereas referencing information is stored in the stack memory. The stack memory is known as Temporary memory allocation.

Memory leakage can occur in the heap memory allocation. The stack memory and heap memory always move close to each other so that they can take up the space of unallocated memory; here, we need to be careful so that they don't collide; otherwise, data will be overwritten. Each task's memory space is allotted, containing Task Control Block and Stack memory. The Task Control Block stores information like priorities and local stack variables. Here the information concerning the task is stored, and the local variable created during the function is stored in the local stack only. It is imperative to precompute the expected stack usage of the task, and it needs to be included in the `xTaskCreate()` function. The kernel object space is used for the queue and semaphores execution.

d) Communication and Synchronisation in RTOS

Task synchronize and communicate with each other using inter-task primitives, kernel objects that facilitate synchronization and communication between two or more threads of execution. Examples of such objects include queues, semaphores and many more such types of objects. About the queue, an explanation will be going to be given further. A queue in an RTOS is a kernel object that allows the

information to be passed [6] without being overwritten by other tasks or getting into the race condition. A Race condition is an error that occurs in any software due to unexpected results because the timing or sequence of the task's actions is affected due to some event or other task [8].

The First In, First Out (FIFO) system is used in a queue, where a value is removed once read. To avoid the overwriting of the values, in the FIFO system, atomic operations like read and write are only executed and cannot be interrupted by other tasks or actions. In a FreeRTOS, the information is copied into values, and while sending some data, the `xQueueSend()` function is applied, where the information is atomically copied to the queue [9]. An example is shown in the queue in Fig. 3. Here Task 1 is executed, and then due to FIFO, Task 1 is executed 4 times, and each value is stored in the queue in a sequence. Then, Task 3 is executed, and each value is stored in the queue. Task B first receives Value 1, then Value 2 and like this, goes to Value 6. This is only called First In, First Out (FIFO). The receive sequence in Fig. 3 is shown using dashed lines. The queues can be very helpful for inter-task communication, sending a message between two tasks [9]. Here, we require enough memory space for the queue to store it as per our needs.

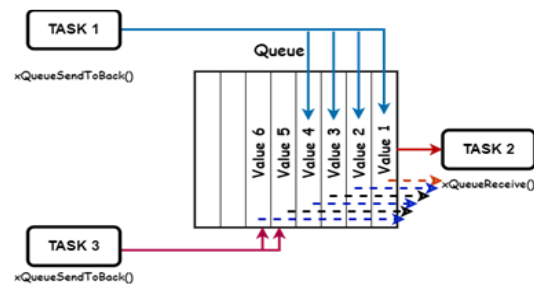


Fig. 3. Task addition in a Queue

e) Management of Timer in RTOS

The timer management function group allows you to create and control timers and call-back functions in a system. Using software timers, we can delay the execution or execute a function periodically. A timer should be created before execution. Whenever the timer gets expires, to call a timer, the call-back function is used [10].

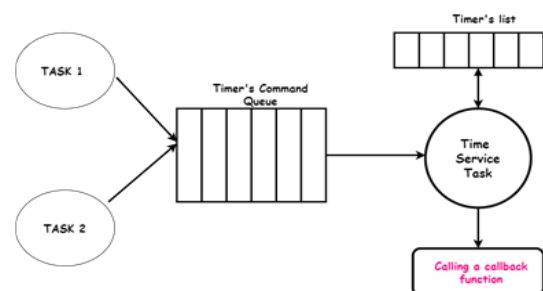


Fig. 4. Block diagram of a software timer

Using commands from the FreeRTOS API for software timers, we can call tasks and control a timer command queue. So, explaining Fig. 4, as we include the FreeRTOS library [1] for the timer, a Timer Service Task is started to run in the background. It is essential to maintain the list of timers and, whenever the timer expires, call them using the call-back function. The software timers are respected as ISR, meaning they cannot be blocked and executed quickly. The block diagram of the software timer can be seen in Fig. 4.

f) *Device Input/Output Management in RTOS*

The Device Input Output management manages the input and output of the various devices. It manages the data exchange between devices with the main memory or the CPU registers. So, I/O modules provide internal interfaces that connect to the central processor and main memory and external interfaces that connect to external devices or peripherals.

B. *Hardware*

For multitasking four stepper motors using RTOS, the material that we are using are ESP32, four units of stepper motors, four units of A4988 stepper motor drivers and a battery to give the power to the Stepper Motor.

a) *Stepper Motor:*

A stepper motor is an electric motor whose main characteristic is that the shaft rotates in steps. It divides a full rotation into steps. The motor's internal structure enables the function, which allows us to know the exact angular position of the shaft by simply counting the number of steps. This property makes it suitable for a wide range of applications. The stepper motor has a fixed part called a stator and a rotating part called a rotor [11]. The teeth with wired coils are called stators. The rotor is either a permanent magnet or a variable reluctance core. The stepper motor works by energizing one or more of the stator phases. The current through the coil produces a magnetic field to which the rotor is aligned. So, by supplying the different phases one after the other, the rotor can be rotated by a fixed amount to reach the desired final position. The stepper motor is used in the R&A industry for various work like 3D printing, pick and place and many more.

b) *Stepper Motor Driver:*

A stepper motor is controlled by a driver that sends a pulse to the motor to rotate it [12]. The A4988 is a micro-stepping driver for controlling bipolar stepper motors with a built-in translator for easy operation. This means the user can control the stepper motor with just two pins from the controller. One pin is there for controlling the steps, and another for controlling the rotation direction.

c) *The Architecture of ESP32:*

In Fig. 5, the block diagram of the ESP32 microcontroller can be seen. A dual-core or single-core supported Xtensa 32-bit LX-6 microprocessor can be found. The 448 Kbytes ROM is there for booting and doing the core functions. ESP-32 consists of 520 Kbytes System-On-Chip RAM for instruction [13]. Here Digital to Analog Converter (DAC) and Analog to Digital Converter (ADC) slots are in ESP32. It consists of a temperature sensor with an internal temperature range of -400 C to +1250 C and a Touch sensor [13].

ESP32 supports Bluetooth and Wireless Connectivity on the chip only. On the ESP32 chip, the Radio block can also be found for wireless connectivity. In this block, only the information is transmitted and received [14]. The ESP32 Ultra-Low Power (ULP) coprocessor is used for doing the ADC in the deep sleep low-power mode. The operating frequency of ESP32 is 240 MHz. The Recovery memory is like SRAM in the RTC, used when the system is in deep sleep mode. The code is stored in the embedded flash memory; internally, it can go up to 4 MB, the external embedded flash memory up to 16 MB, and SRAM up to 8 MB can be attached. There is a block for Cryptographic Hardware Acceleration. This block is responsible for encryption and decryption, increasing the computing speed and is essential for preventing

hacking. In ESP32, Serial Peripheral Interface (SPI) is used for synchronous communication, and Universal Asynchronous Receiver-Transmitter (UART) is used for asynchronous communication between two devices. There is Pulse Width Modulation (PWM) port which able to support analogue outputs. Inter-Integrated Circuit (I2C) in ESP-32 is a bus interface protocol for serial communication [13]. It supports the master and slave modes. The master is the device that is driving the SCL, and the slave is responding to the master.

In Fig. 6, the pinout of ESP32 is shown. Here there are 25 General Purpose Input and Output (GPIO) pins. The GPIO pins are the standard interface to connect the microcontroller to other devices. There are two pins for ground and 5V and 3.3V volt power supply pins. All pins that can work as output can be used as PWM, as seen in Fig. 7. In the ESP32 pinout diagram for 30 pins, there are 15 Analog to Digital Converter (ADC) pins. Here, we convert an analogue to a digital signal. In ESP32, the pinout diagram for 30 pins, there are 2 DAC pins; here, we convert a digital to an analog signal.

C. *The Architecture of the Proposed System*

The connections are done in the simulator and can be seen in Fig. 7, and in Table 2, it can be seen in tabular format. The pin diagram of the A4988 stepper motor driver can be seen in Fig. 9. In A4988 Stepper Motor Driver, the GND and VMOT are the power pins. The MS1/2/3 are there for dividing a single step into smaller steps by setting the logic levels thrice. Pulling the SLEEP (SLP) pin LOW allows the driver to get into sleep mode, reducing power consumption. Pulling the RESET (RST) pin LOW, the driver ignores the STEP inputs and resets the driver. The ENABLE (EN) pin is, by default, LOW, which means it is enabled. It is essential when we need to stop for any emergency by making it HIGH. The STEP pin is responsible for controlling the micro-steps of the stepper motor, and the DIR pin is responsible for the direction of the Stepper motor by making DIR HIGH. It moves clockwise and counter-clockwise by making it LOW. The 1A, 1B, 2A, and 2B are output channels of the A4988 motor driver. They are going to connect with the stepper motor [15].

The output pins from the A4988 Stepper motor driver, i.e., 1A, 1B, 2A, 2B, will be input for the stepper motor [11]. It can be seen in the circuit diagram in Fig. 7. The four pins of the stepper motor can be seen in Fig. 8. The output pins will be connected to the driver in these four pins. So, according to the connection in Fig. 7, multitasking the four stepper motors can be done. The simulation is done in the Wokwi simulator.

D. *RTOS Code Architecture for Proposed System*

For Hardware simulation, the FreeRTOS library must be installed in Arduino IDE; for software simulation, the FreeRTOS library must be installed in the Wokwi simulator [1]. In Fig. 10, the flowchart of the code is shown. Explaining it,

- *STEP - 1: Starting the code.*
- *STEP - 2: Including the FreeRTOS library.*
- *STEP - 3: Deciding or defining which core execution needs to be done, which means single core or dual core.*
- *STEP - 4: Defining the ENABLE (EN), STEP (STP) and DIR pins. Code needs to be included in the structure of stepping orders.*

- **STEP - 5:** Write the code for multitasking or without multitasking by using the functions. The code is there for double-motor multitasking, which has 6 combinations, triple- motor multitasking, which has 3 combinations and four-motor multitasking. As we know, there are 4 Stepper motors here.
- **STEP - 6:** Writing the Code for Software Time. The algorithm for this can be seen further.
- **STEP - 7:** Writing the setup code, in that block GPIO setup, creating a one-shot timer, and multitasking setup is included.
- **STEP – 8:** The execution needs to be done by calling the function in the loop so that it can run repeatedly.
- **STEP – 9:** STOP.

Fig. 10 depicts the algorithm for the Software timer, which is described below,

The time is always in Tick timer format. The conversion into the millisecond needs to be done.

- **STEP – 1:** Make the call-back function for the software timer.
- **STEP – 2:** Write the code for doing action when input is done in the serial input. Here, stepper motors 1 and 3 stop for 5 seconds and then resume the rotation.
- **STEP – 3:** Check whether there are things in the serial buffer. If yes, go to STEP-4; otherwise, go to STEP-5
- **STEP – 4:** Print back the serial monitor number and do the inputted action using the code in STEP-3.
- **STEP – 5:** Continue further.

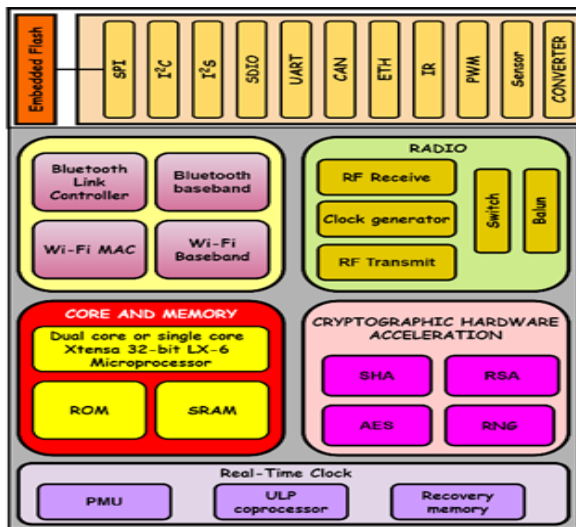


Fig. 5. Architectural Block Diagram of ESP2

TABLE II-CIRCUIT DIAGRAM CONNECTION TABLE

Stepper Motor (STEPPER) with No. and Stepper Motor Driver (DRV) with No.	Stepper Motor Pins	A4988 Stepper Motor Driver Pins	ESP32 Pinout (30 Pins)	Battery Pins
STEPPER & DRV-1,2,3,4	A-	1B		
STEPPER & DRV-1,2,3,4	A+	1A		
STEPPER & DRV-1,2,3,4	B+	2A		
STEPPER & DRV-1,2,3,4	B-	2B		

DRV- 1,2,3,4		MS1	3.3V	
DRV- 1,2,3,4		MS2	3.3V	
DRV- 1,2,3,4		MS3	3.3V	
DRV- 1,2,3,4		RESE T	3.3V	
DRV- 1,2,3,4		SLEE P	3.3V	
DRV- 1		ENAB LE (EN)	21	
DRV- 1		STEP	23	
DRV- 1		DIR	22	
DRV- 2		ENAB LE (EN)	5	
DRV- 2		STEP	19	
DRV- 2		DIR	18	
DRV- 3		ENAB LE (EN)	26	
DRV- 3		STEP	27	
DRV- 3		DIR	14	
DRV- 4		ENAB LE (EN)	32	
DRV- 4		STEP	33	
DRV- 4		DIR	25	
DRV- 1,2,3,4		VMO T		VCC
DRV- 1,2,3,4		GND	GND	GND

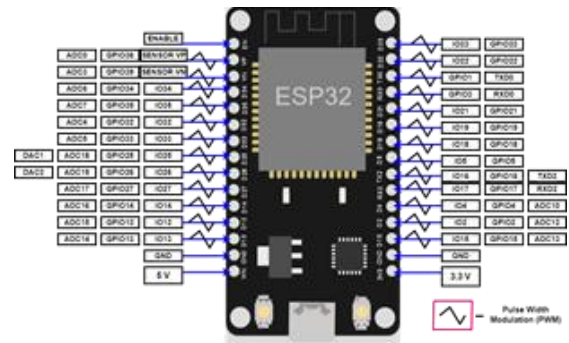


FIG. 6. ESP32 Pinout Diagram

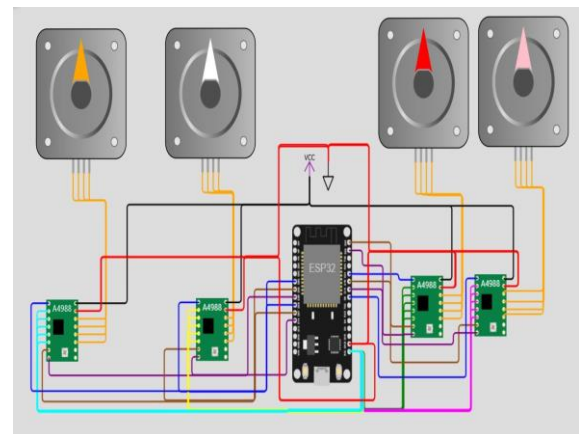


Fig. 7. Circuit Diagram in a Simulator

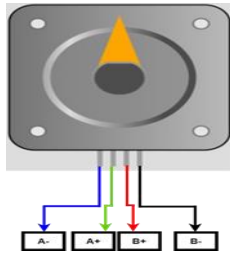


Fig. 8. Stepper motor pin diagram in Wokwi simulator

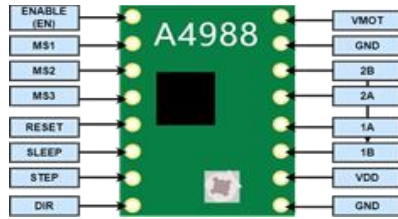


Fig. 9. A4988 Stepper motor driver pin diagram

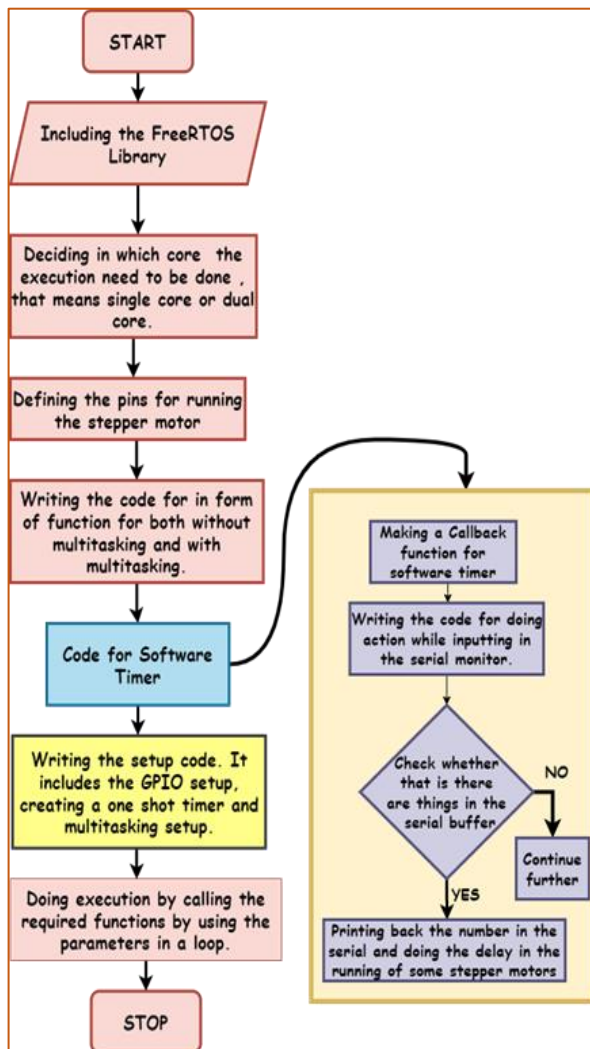


Fig. 10. Flowchart of RTOS code for multitasking of four stepper motors

For multitasking, the code for the queue needs to be written in starting a code to handle the stepper motor for multitasking using `xQueueHandle`, which is used as a queue pointer. The code is for double-motor multitasking with 6 combinations, triple-motor multitasking with 3 combinations

and four-motor multitasking. For double-motor multitasking, we need to send the two-motor stepping order into the queue and wait for all motor bits as a sync indicator.

Giving an example of two-motor multitasking here, we define the function and take the parameters like steps and delay in milliseconds for both motors. The structure we had made for multitasking before, here we call it and send the parameters to it. Then, use `xQueueSendToBack` append or put the value in the queue from the back side. So, as per FIFO, we receive the first value, which is inputted first and will only be executed as discussed in point A.

Hereafter, we wait for all the bits. The `xEventGroupWaitBits` reads a bit in an RTOS event group and optionally enters a blocking state (with timeout), waiting for the bit and bit group to be set. And if the bits are got equal to either of both, then the two motors get synced. Otherwise, we check if there are no more messages, then we take a break and print in the serial monitor that the two motors are waiting. Here in the `xEventGroupWaitBits`, the maximum wait is done for 60 seconds. The parameters and conditions for `xEventGroupBits` must be changed for three and four-motor multitasking. Also, according to the stepper motor number, we need to change the motor number in the code for that respective function. This is done in an infinite loop using the while (1) command.

E. Proposed Methodology

Firstly, we need to make the connection as per the connection diagram and code it accordingly. When the simulation starts, we call the function by giving the parameters as required for action to do the task for various works. So, whenever the function called the required stepper motor driver gets current, the phase control logic is executed, and the pulse is sent to the motor to rotate the stepper motor. The driver includes the power amplifier, which sends the motor current to the stepper motor. When the stepper motor is running, if the character is inputted in the serial, then stepper motors 1 and 3 stop for 5 seconds and after completion of 5 s, the motor resumes running. The functions are called sequentially. It can be for single, double, triple or four-stepper motor multitasking as per our needs. This sequential manner will repeat itself in an infinite loop. So, our required tasks can repeatedly run without human intervention.

III. RESULT AND DISCUSSION

In Fig. 12, the multitasking of four stepper motors is shown. As the RTOS code is loaded on the ESP32 for multitasking the four stepper motor, the stepper motor driver passes the motor current so all four steppers can run. Here, stepper motors 1 and 3 run in the counter-clockwise direction and stepper motors 2 and 4 in the clockwise direction. There are 14 possibilities for four stepper motors, which include 4 possibilities for motors that can run independently, 6 possibilities for two-motor multitasking, 3 possibilities for three-motor multitasking and 1 possibility of four-motor multitasking are implemented. Multiple possibilities functions are called sequentially and put in an infinite loop. Here while switching between the task, the delay is 150 ms. This multitasking of the four stepper motors is done, and the machines and arms in the R&A industry can be used for translation and rotation by giving the proper step parameters and required delay in milliseconds.

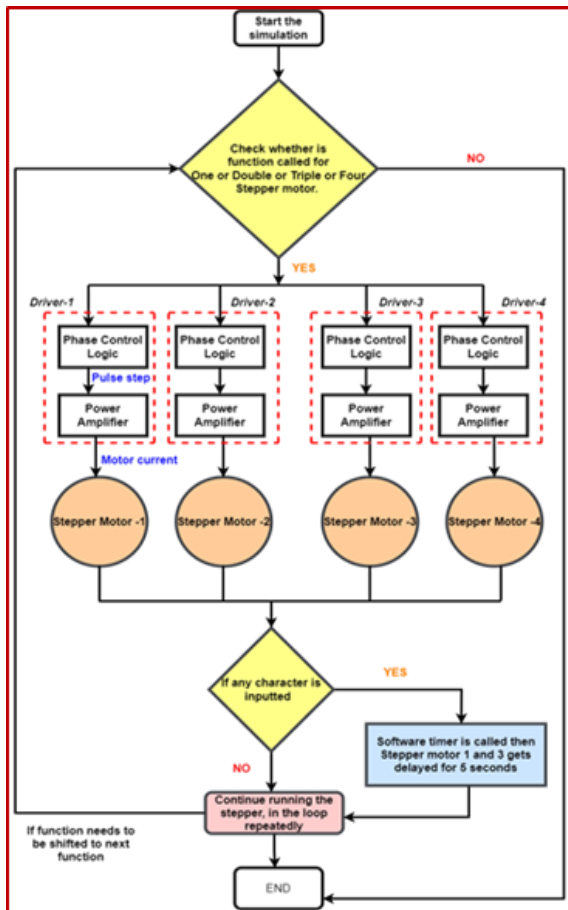


Fig. 11. Flowchart of methodology for multitasking of four stepper motors

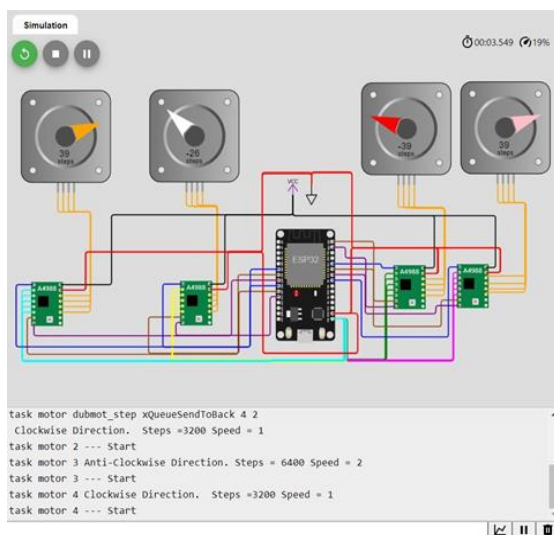


Fig. 12. Simulation running for multitasking of four stepper motors

In Fig. 13, when we inputted the value 5 in the serial monitor, the steppers 1 and 3 stopped at the 20th step and 30th step in the anti-clockwise direction at 3 seconds. Here the software timer is invoked when we input the value in the serial monitor as we defined it while programming.

It is visible in Fig. 14. After 5 seconds, the stepper motor 1 and 3 resume running in the clockwise direction, which means that the software timer is revoked. If we invoked the software timer once more, the stepper motor started running

in the clockwise direction after revoking the software timer. Such results are found while doing the simulation.

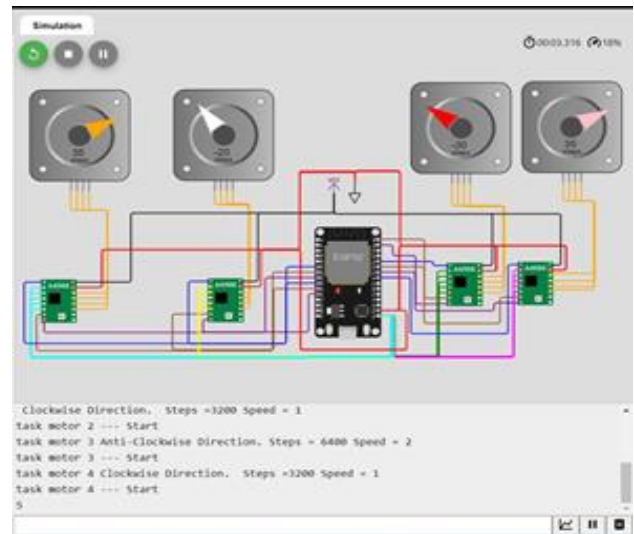


Fig. 13. Software timer is invoked

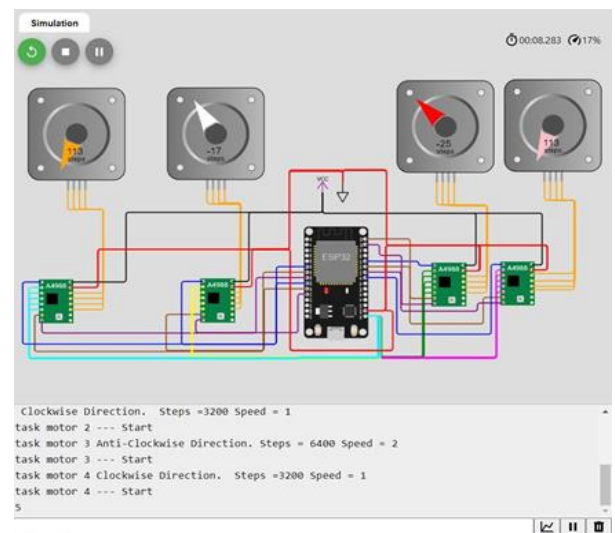


Fig. 14. Software timer is revoked after 5 seconds

IV. CONCLUSION

In this paper, we have presented the FreeRTOS, a Real-Time Operating System (RTOS) kernel. The integration of the FreeRTOS, ESP32 and Stepper motor is done. The multitasking of four stepper motors with 14 possibilities is implemented. Here we can also run various possibilities sequentially for infinite times in a loop. Also, the delay between switching functions is 150 ms, much less than other OS. Here we also presented the queue and software timer. By comparing the [17] research paper, our system performed better and produced a better result than [17]. In [17] comparison of the different OS and FreeRTOS response or delay, our system is far better than Windows. From [16], our proposed system is a bit less costly and practical. The queue implementation is done for multitasking, and a software timer is also implemented to delay a task. The development in this can be seen by making it wirelessly monitored. The cost of the proposed system is also very low compared to the systems that are using the industries. Multitasking four steppers became very efficient, and due to little task switching time,

the motors performed better and smoothly. Compared to the research paper, the proposed system for multitasking four stepper motors is done very well.

REFERENCES

- [1] R. Barry. [Online]. Available: https://www.freertos.org/fr-content-src/uploads/2018/07/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf
- [2] J. A. Stankovic and R. Rajkumar, "Real-time operating systems," *Real-Time Systems*, vol. 28, no. 2-3, pp. 237–253, 2004.
- [3] M. A. Rahman, Y. Li, T. Nabeed, and M. T. Rahman, "Remote monitoring of heart rate and ECG signal using ESP32," 2021 4th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE), pp. 604–610, 2021.
- [4] S. Debbarma and K. Chakma, "An approach to advance real-time os services with soft-error," *International Journal of Computer and Electrical Engineering*, vol. 3, no. 4, pp. 548–548, 2011.
- [5] Wilmshurst, T. (2006). *Designing embedded systems with PIC microcontrollers: principles and applications*. Elsevier.
- [6] D. Déharbe, S. Galvao, and A. M. Moreira, "Formalizing freertos: First steps," in *Brazilian Symposium on Formal Methods*. Springer, 2009, pp. 101–117.
- [7] D. Ibrahim, *ARM-Based Microcontroller Multitasking Projects: Using the FreeRTOS Multitasking Kernel*, Newnes, 2020.
- [8] S. Carr, J. Mayo, and C. K. Shene, "Race conditions: a case study," *Journal of Computing Sciences in Colleges*, vol. 17, no. 1, pp. 90–105, 2001.
- [9] M. Y. Zhu, *Understanding FreeRTOS: A requirement analysis*. Beijing, China, Tech. Rep: CoreTek Syst., Inc, 2016.
- [10] Ibrahim, D. (2020). *ARM-Based Microcontroller Multitasking Projects: Using the FreeRTOS Multitasking Kernel*. Newnes.
- [11] T. S. Roy, H. Kabir, and M. A. M. Chowdhury, "Simple Discussion on Stepper Motors for the Development of Electronic Device," *International Journal of Scientific & Engineering Research*, vol. 5, no. 1, pp. 1089– 1096, 2014.
- [12] X. Wang, "Research on Stepper Motor Control System based on Single Chip Microcomputer," 4th International Conference on Electrical & Electronics Engineering and Computer Science (ICEEECS 2016), pp. 105–108, 2016.
- [13] A. Maier, A. Sharp, and Y. Vagapov, "Comparative analysis and practical implementation of the ESP32 microcontroller module for the internet of things," 2017 Internet Technologies and Applications (ITA), pp. 143– 148, 2017.
- [14] M. Atif, S. Muralidharan, H. Ko, and B. Yoo, "Wi-ESP-a tool for CSI-based device-free Wi-Fi sensing (DFWS)," *Journal of Computational Design and Engineering*, vol. 7, no. 5, pp. 644–656, 2020.
- [15] R. Varga, T. I. Erdei, and G. Husi, "CodeSys based HMI design and sensor upgrades for a PLC and Atmega2560 controlled industrial conveyor," *IOP Conference Series: Materials Science and Engineering*, vol. 568, pp. 12 062–12 062, 2019.
- [16] Ungurean, I. (2020). Timing comparison of the real-time operating systems for small microcontrollers. *Symmetry*, 12(4), 592.
- [17] Abdulganiyu, A., & Rabi, I. (2017). Comparative Analysis of Real-Time Operating System (RTOS) of Some Selected OS Using External Signal Generator and Oscilloscope. *International Journal of Science and Engineering Investigations*, 6(63), 47-53.
- [18] Bogdan, L. (2011). PLC as a driver for stepper motor control. *Annals of the Oradea University, Fascicle of Management and Technological Engineering*, 10, 3-9.
- [19] Azad, A. M., Amin, A. A., Faruk, A. A., & Alam, M. A. (2009, January). Stepper motor performance under real-time multitasking environment. In *TENCON 2009-2009 IEEE Region 10 Conference* (pp. 1-6). IEEE.